

Section 97 Editorial — Digit And Broken-Profile DP

A. Digit Sum Range

How to find it

Use a tight digit DP and range subtraction.

Correctness

The state enumerates every bound-respecting prefix once, and the base accepts exactly the desired sum. Subtracting prefixes leaves precisely the interval.

Complexity

$O(19 * S * 10)$ per query.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
string s;
ll memo[20][172];
bool seen[20][172];
int target;
ll dfs(int pos,int sum,bool tight) {
    if(sum>target)return 0;
    if(pos==(int)s.size())return sum==target;
    if(!tight&&seen[pos][sum])return memo[pos][sum];
    int lim=tight?s[pos]-'0':9;
    ll ans=0;
    for(int d=0;d<=lim;d++)ans+=dfs(pos+1,sum+d,tight&&d==lim);
    if(!tight)seen[pos][sum]=1,memo[pos][sum]=ans;
    return ans;
}
ll count_to(ll n,int wanted) {
    if(n<0)return 0;
    s=to_string(n);
    target=wanted;
    memset(seen,0,sizeof(seen));
    return dfs(0,0,1);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin>>q;
    while(q--) {
        ll l,r;
        int x;
        cin>>l>>r>>x;
```

```

        cout<<count_to(r,x)-count_to(l-1,x)<<'\n';
    }
}

```

Python

```

import sys
from functools import lru_cache

def count(n, target):
    if n < 0:
        return 0
    digits = list(map(int, str(n)))

    @lru_cache(None)
    def dp(pos, total, tight):
        if total > target:
            return 0
        if pos == len(digits):
            return int(total == target)
        limit = digits[pos] if tight else 9
        return sum((dp(pos + 1, total + d, tight and d == limit) for d in range(limit + 1)))
    return dp(0, 0, True)

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    print('\n'.join((str(count(d[i + 1], d[i + 2]) - count(d[i] - 1, d[i + 2])) for i in range(1, len(d), 3))))
if __name__ == '__main__':
    main()

```

B. Forbidden Decimal Pattern

How to find it

Take the product of digit-bound state and a KMP prefix automaton.

Correctness

The KMP state exactly summarizes every suffix relevant to a future pattern occurrence; rejecting the terminal state is necessary and sufficient.

Complexity

$O(19 * |P| * 10)$ per query.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll=long long;
string digits,pat;
vector<int>fail;
map<tuple<int,int,bool,bool>,ll>memo;
int step(int j,int d) {
    char c='0'+d;
    while(j&&pat[j]!=c)j=fail[j-1];
    if(pat[j]==c)j++;
}

```

```

    return j;
}
ll dfs(int pos,int j,bool tight,bool started) {
    if(pos==(int)digits.size())return started||pat!="0";
    auto key=tuple {
        pos,j,tight,started
    }
    ;
    if(!tight&&memo.count(key))return memo[key];
    int lim=tight?digits[pos]-'0':9;
    ll ans=0;
    for(int d=0;d<=lim;d++) {
        bool nt=tight&&d==lim;
        if(!started&&d==0)ans+=dfs(pos+1,0,nt,0);
        else {
            int nj=step(j,d);
            if(nj<(int)pat.size())ans+=dfs(pos+1,nj,nt,1);
        }
    }
    if(!tight)memo[key]=ans;
    return ans;
}
ll solve(ll n,string p) {
    digits=to_string(n);
    pat=p;
    fail.assign(p.size(),0);
    for(int i=1;i<(int)p.size();i++) {
        int j=fail[i-1];
        while(j&&p[i]!=p[j])j=fail[j-1];
        if(p[i]==p[j])j++;
        fail[i]=j;
    }
    memo.clear();
    return dfs(0,0,1,0);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin>>q;
    while(q--) {
        ll n;
        string p;
        cin>>n>>p;
        cout<<solve(n,p)<<'\\n';
    }
}

```

Python

```

import sys
from functools import lru_cache

def count(n, pattern):
    digits = list(map(int, str(n)))
    p = list(map(int, pattern))
    fail = [0] * len(p)
    for i in range(1, len(p)):
        j = fail[i - 1]
        while j and p[i] != p[j]:
            j = fail[j - 1]
        if p[i] == p[j]:

```

```

        j += 1
        fail[i] = j

def step(j, d):
    while j and p[j] != d:
        j = fail[j - 1]
    if p[j] == d:
        j += 1
    return j

@lru_cache(None)
def dp(pos, j, tight, started):
    if pos == len(digits):
        return int(started or pattern != '0')
    limit = digits[pos] if tight else 9
    answer = 0
    for d in range(limit + 1):
        nt = tight and d == limit
        if not started and d == 0:
            answer += dp(pos + 1, 0, nt, False)
        else:
            nj = step(j, d)
            if nj < len(p):
                answer += dp(pos + 1, nj, nt, True)
    return answer
return dp(0, 0, True, False)

def main():
    data = sys.stdin.buffer.read().split()
    out = []
    for i in range(1, len(data), 2):
        out.append(str(count(int(data[i]), data[i + 1].decode()))))
    print('\n'.join(out))
if __name__ == '__main__':
    main()

```

C. Obstacle Domino Tilings

How to find it

Scan the first uncovered cell under an incoming occupancy mask.

Correctness

Every tiling covers that cell horizontally or vertically exactly once, and each legal transition preserves nonoverlap and complete coverage.

Complexity

$O(n * 2^m * m)$ up to transition constants.

C++

```

#include <bits/stdc++.h>
using namespace std;
const int MOD=1e9+7;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int n,m;
cin>>n>>m;
vector<string>g(n);
for(auto&s:g)cin>>s;
vector<int>dp(1<<m);
dp[0]=1;
for(int r=0;r<n;r++) {
    int blocked=0;
    for(int c=0;c<m;c++)if(g[r][c]=='#')blocked|=1<<c;
    vector<int>nd(1<<m);
    for(int in=0;in<1<<m;in++)if(dp[in]&&!(in&blocked)) {
        function<void(int,int,int)>fill=[&](int c,int used,int out) {
            if(c==m) {
                nd[out]=(nd[out]+dp[in])%MOD;
                return;
            }
            int bit=1<<c;
            if((used|blocked)&bit)fill(c+1,used,out);
            else {
                if(c+1<m&&!((used|blocked)&(bit<<1)))fill(c+2,used|bit|(bit<<1),out);
                if(r+1<n&&g[r+1][c]=='.')fill(c+1,used|bit,out|bit);
            }
        };
        fill(0,in,0);
    }
    dp.swap(nd);
}
cout<<dp[0]<<'\n';
}

```

Python

```

import sys
MOD = 1000000007

def main():
    data = sys.stdin.buffer.read().split()
    n, m = map(int, data[:2])
    grid = [x.decode() for x in data[2:]]
    dp = {0: 1}
    for row in range(n):
        blocked = sum(((grid[row][c] == '#') << c for c in range(m)))
        nd = {}
        for incoming, value in dp.items():
            if incoming & blocked:
                continue

            def fill(c, used, out):
                if c == m:
                    nd[out] = (nd.get(out, 0) + value) % MOD
                    return
                bit = 1 << c
                if (used | blocked) & bit:
                    fill(c + 1, used, out)
                else:
                    if c + 1 < m and (not (used | blocked) >> c + 1 & 1):
                        fill(c + 2, used | bit | bit << 1, out)
                    if row + 1 < n and grid[row + 1][c] == '.':
                        fill(c + 1, used | bit, out | bit)
            fill(0, incoming, 0)
        dp = nd

```

```

    print(dp.get(0, 0))
if __name__ == '__main__':
    main()

```

D. Grid Independent Sets

How to find it

Enumerate horizontally valid row masks and reject overlap with blocked cells or the previous row.

Correctness

These tests are exactly the horizontal and vertical adjacency constraints, so row induction is bijective.

Complexity

$O(n * V^2)$, where V is the number of horizontally valid masks.

C++

```

#include <bits/stdc++.h>
using namespace std;
const int MOD=1e9+7;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,m;
    cin>>n>>m;
    vector<string>g(n);
    for(auto&s:g)cin>>s;
    vector<int>valid;
    for(int x=0;x<1<<m;x++)if(!(x&(x<<1)))valid.push_back(x);
    vector<int>dp(1<<m);
    dp[0]=1;
    for(auto row:g) {
        int blocked=0;
        for(int c=0;c<m;c++)if(row[c]=='#')blocked|=1<<c;
        vector<int>nd(1<<m);
        for(int x:valid)if(!(x&blocked))for(int y:valid)if(!(x&y))nd[x]=(nd[x]+dp[y])%MOD;
        dp.swap(nd);
    }
    cout<<accumulate(dp.begin(),dp.end(),0LL)%MOD<<'\\n';
}

```

Python

```

import sys
MOD = 1000000007

def main():
    data = sys.stdin.buffer.read().split()
    n, m = map(int, data[:2])
    grid = [x.decode() for x in data[2:]]
    valid = [x for x in range(1 << m) if not x & x << 1]
    dp = {0: 1}
    for row in grid:
        blocked = sum((row[c] == '#') << c for c in range(m))

```

```

    nd = {}
    for mask in valid:
        if mask & blocked:
            continue
        nd[mask] = sum((value for old, value in dp.items() if not mask & old)) % MOD
    dp = nd
    print(sum(dp.values()) % MOD)
if __name__ == '__main__':
    main()

```

State-minimality and stress companion

For digit DP, two prefixes may merge only when their tightness and every property state relevant to future digits agree. Leading zeroes enter the automaton only if the statement treats them as representation digits. Define the prefix-count function as zero on negative bounds before using range subtraction.

For profile DP, each incoming bit occupies the current cell from above. Scan the first unresolved cell so every placement has a unique transition history. Reject incoming bits on obstacles and require the row to be fully consumed before passing the outgoing mask. Exhaustively enumerate small integers, domino placements, and independent cell subsets; test empty ranges, the number zero, all-blocked rows, and width one.